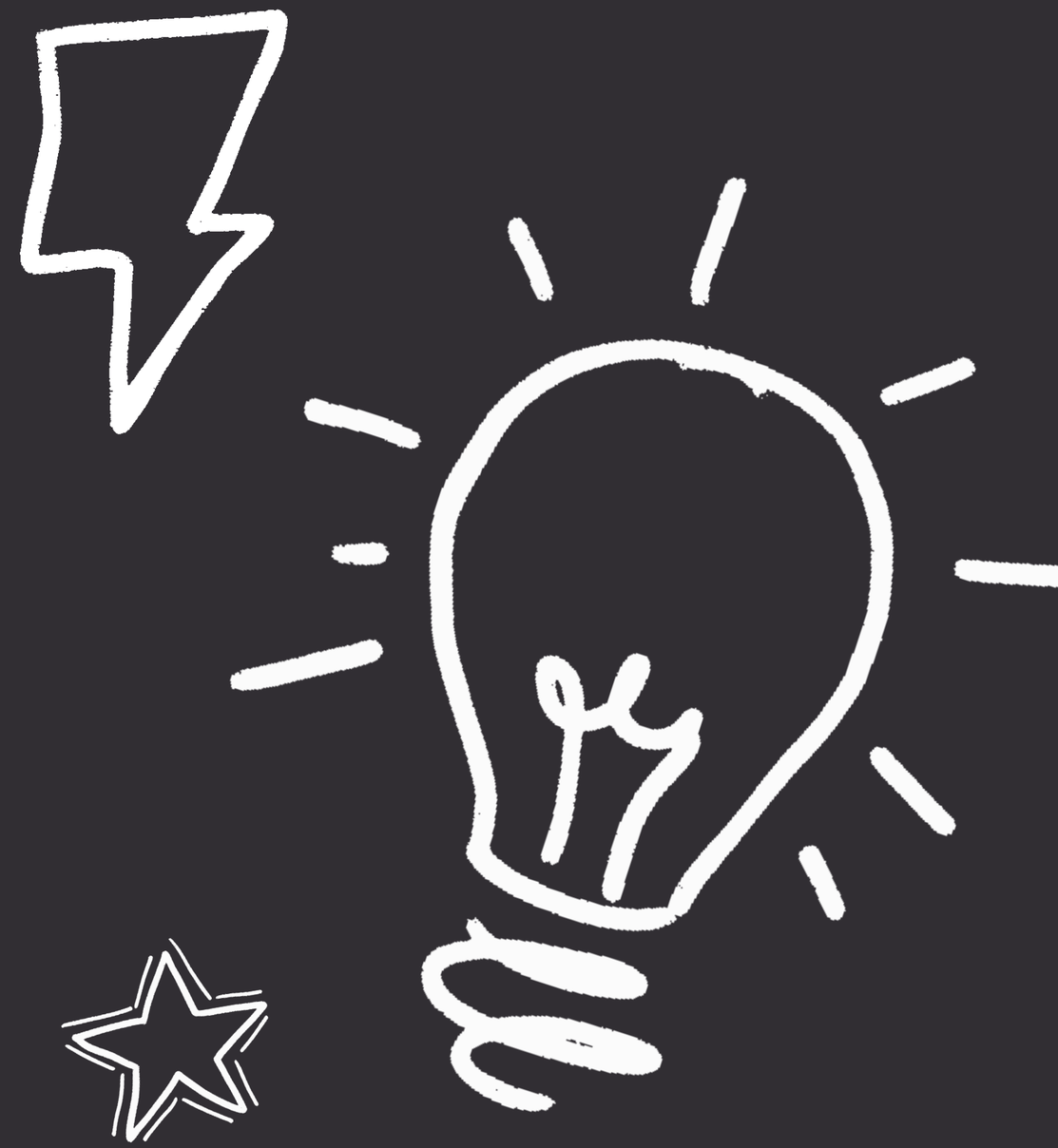
Clase generice si comparatori

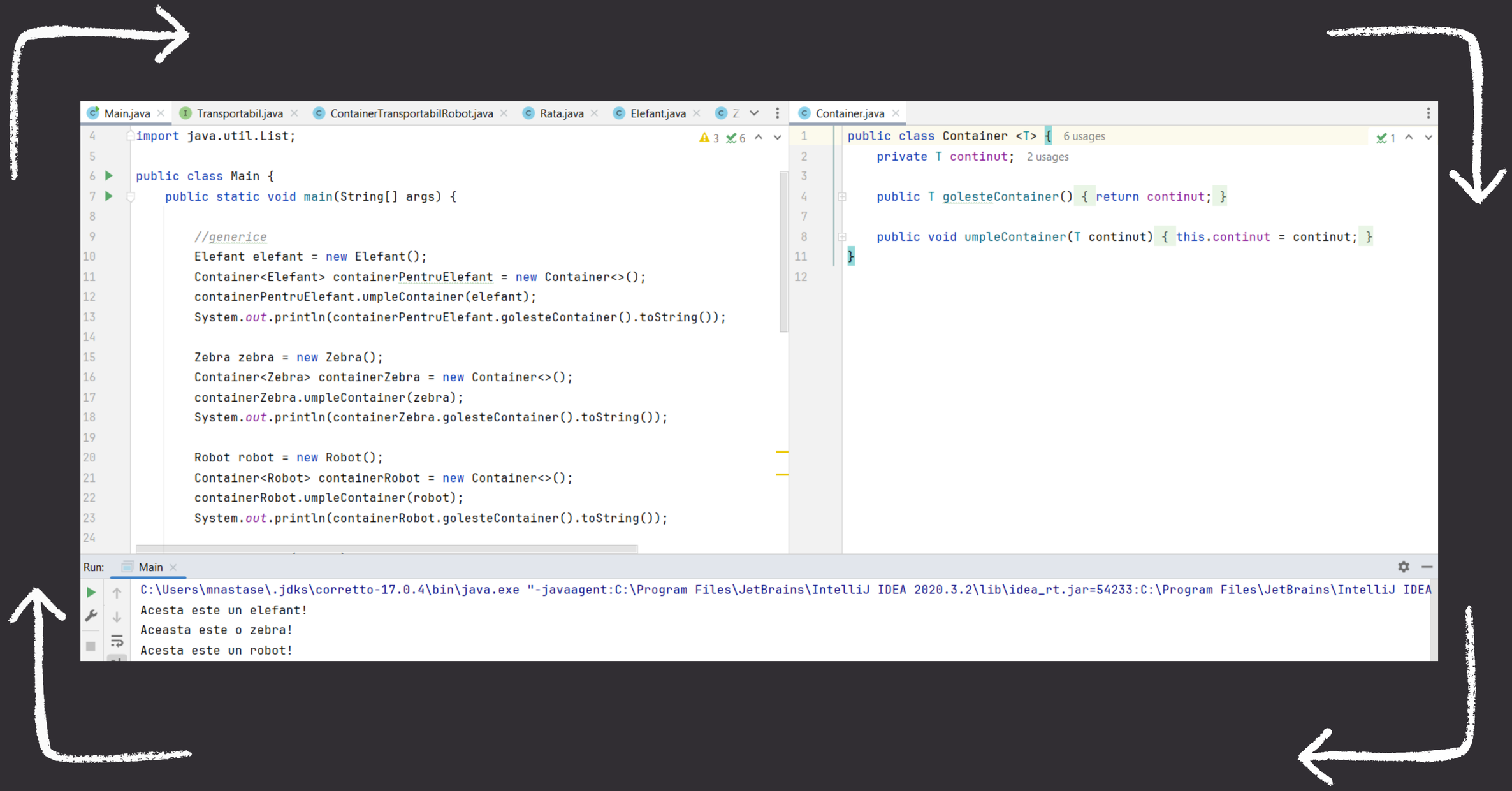


Clasa generica

O clasa generica este o clasa care foloseste un parametru formal (ex: <T>) care poate lua forma a diferite obiecte in functie de implementare.

Scenariu: Dorim crearea unui container in care putem stoca diferite obiecte (Elefant, Zebra, Robot). Pentru fiecare obiect in parte dorim un comportament diferit (Afisarea tipului fiecarui obiect).
Solutie: Se declara o clasa generica Container<T> unde T poate fi oricare dintre obiectele dorite.




Main.java
Transportabil.java
ContainerTransportabilRobot.java
Rata.java
Elefant.java
Container.java
import java.util.List;
public class Main {
public static void main(String[] args) {
//generice
Elefant elefant = new Elefant();
Container<Elefant> containerPentruElefant = new Container<>();
containerPentruElefant.umpleContainer(elefant);
System.out.println(containerPentruElefant.golesteContainer().toString());
Zebra zebra = new Zebra();
Container<Zebra> containerZebra = new Container<>();
containerZebra.umpleContainer(zebra);
System.out.println(containerZebra.golesteContainer().toString());
Robot robot = new Robot();
Container<Robot> containerRobot = new Container<>();
containerRobot.umpleContainer(robot);
System.out.println(containerRobot.golesteContainer().toString());
public class Container <T> { 6 usages
private T continut; 2 usages
public T golesteContainer() { return continut; }
public void umpleContainer(T continut) { this.continut = continut; }
Run:
Main
C:\Users\mnastase\.jdks\corretto-17.0.4\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2020.3.2\lib\idea_rt.jar=54233:C:\Program Files\JetBrains\IntelliJ IDEA
Acesta este un elefant!
Aceasta este o zebra!
Acesta este un robot!

Interfete generice

Dorim crearea unui container care poate transporta diferite obiecte (ex: printandu-le numele).

Solutie: Se creaza interfata generica Transportabil<T> cu o singura metoda transporta care poate fi implementata in functie de tipul obiectului. Se creaza o clasa ContainerTransportabilRobot care implementeaza interfata Transportabil folosind numele obiectului Robot.

Rata.java | Elefant.java | Transportabil.java | Zebra.j

```
1  public interface Transportabil <T>{  1 usage  2
2      void transporta(T t);  1 usage  1 implementation
3  }
4
```

Container.java | ContainerTransportabilRobot.java

```
1  public class ContainerTransportabilRobot implements Transportabil<Robot>{  2 usages
2      @Override  1 usage
3      public void transporta(Robot robot) {
4          System.out.println("Transporta robot "+ robot.getNume());
5      }
6  }
7
```

Main.java

```
6  public class Main {
7      public static void main(String[] args) {
24
25         robot.setNume("Robo");
26         ContainerTransportabilRobot containerTransportRobot = new ContainerTransportabilRobot();
27         containerTransportRobot.transporta(robot);
28
29
```

Compararea obiectelor



Compararea obiectelor

- In multe situatii, este necesara compararea obiectelor complexe (sortare, cautare). Pentru numere, sortarea se face in ordinea numerica, pentru siruri de caractere, se foloseste sortarea alfabetica.
- Pentru sortarea obiectelor mai complexe, este necesara declararea metodelor de sortare, folosind interfetele Comparator sau Comparable.

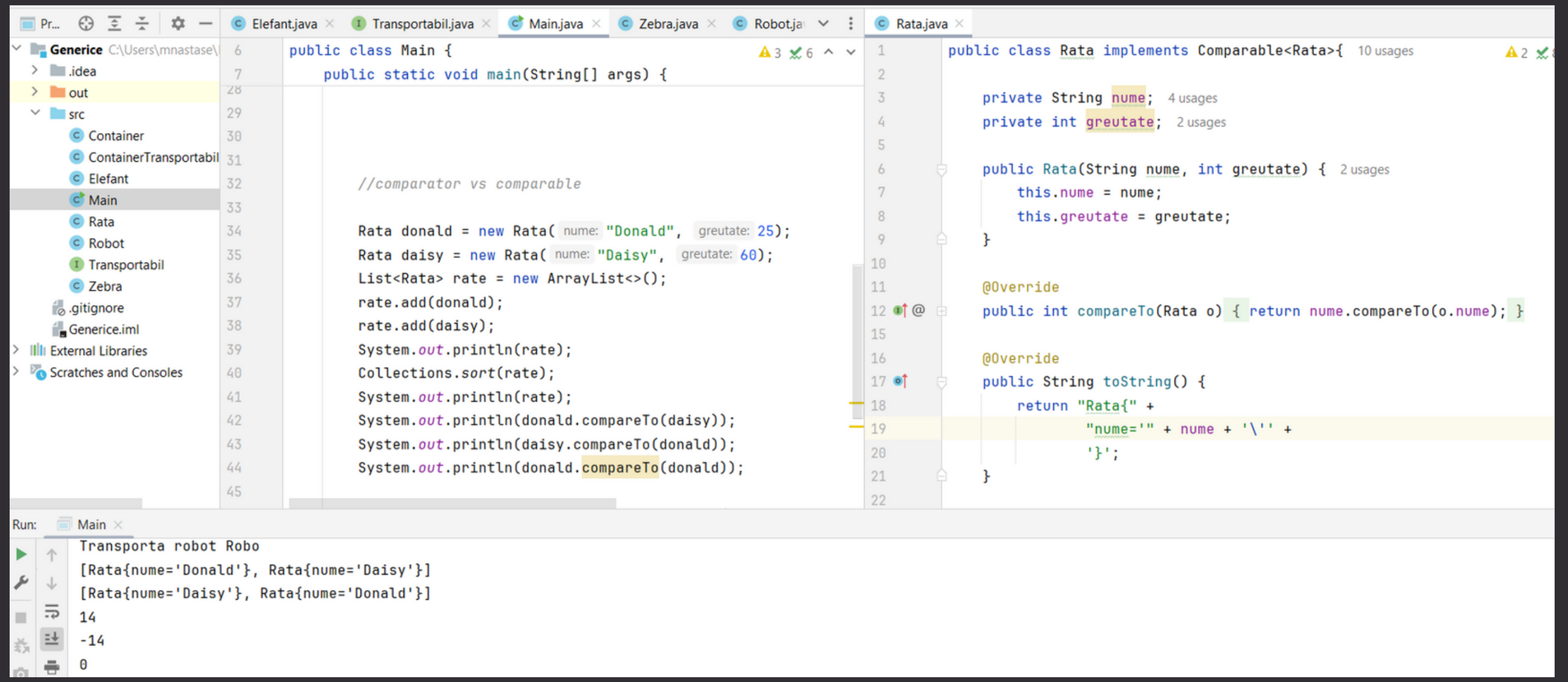
Interfete generice

Crearea unui obiect “comparabil” se poate realiza implementand interfata Comparable si oferind o implementare pentru metoda compareTo.

Exemplu: Pentru clasa rata se doreste compararea dupa numele acestora (prin urmare ordinea alfabetica).

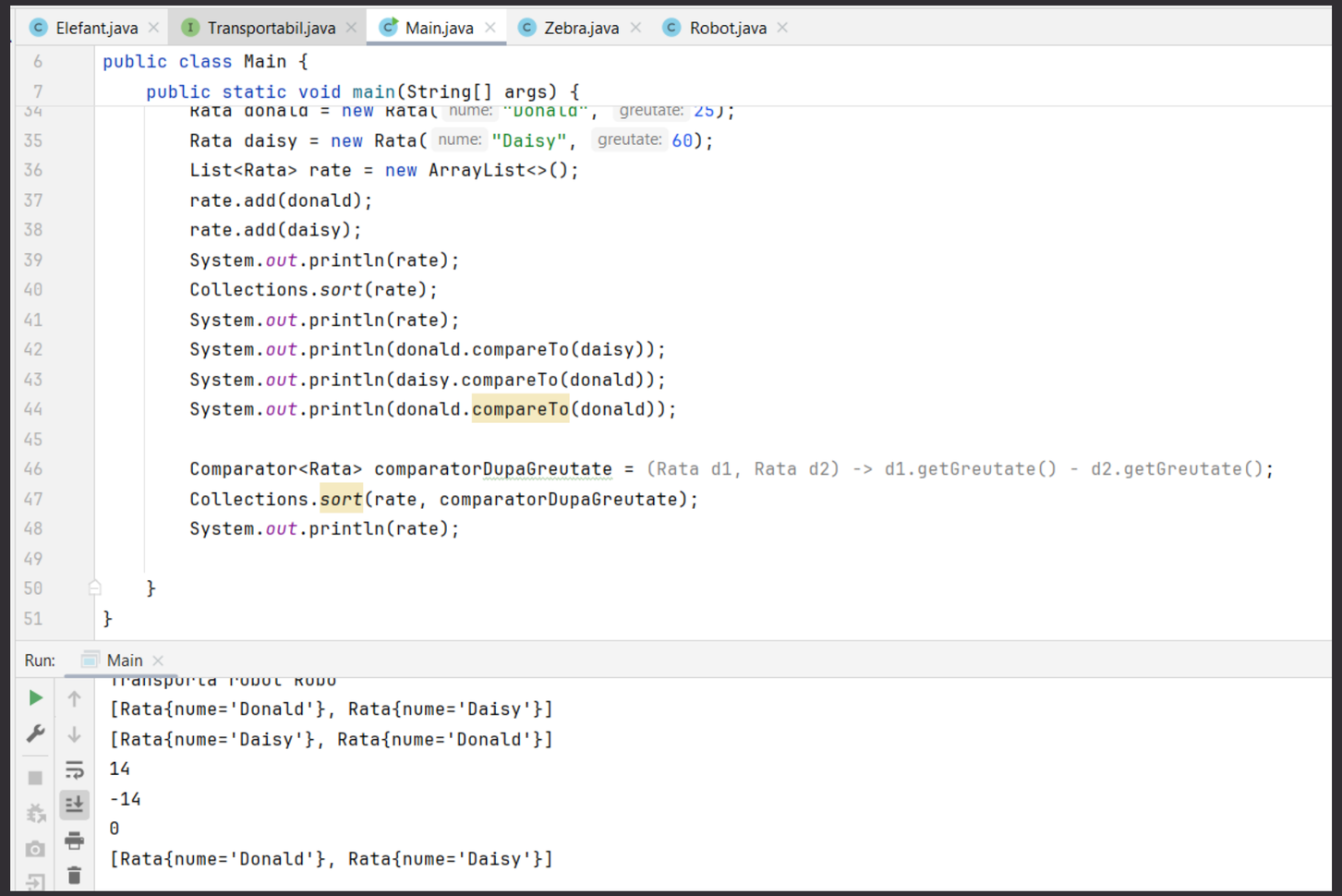
Pentru a demonstra functionarea comparatorului, se pot sorat obiectele de tip Rata folosind metodaCollection.sort() care sorteaza o lista de obiecte folosind implementarea metodei compareTo() din clasa obiectelor din lista.

Metoda compareTo returneaza 0 daca obiectele sunt egale, un numar pozitiv daca primul este mai mic decat al doilea si un numar negativ altfel.



Interfata Comparator

- Compararea obiectelor se poate face si folosind interfata Comparator care are o structura si functionalitat similare cu Comparable.
- Implementarea metodei compare se poate face folosind expresii lambda ca in exemplu.



Resurse

- https://www.geeksforgeeks.org/generic-class-in-java/
- https://www.baeldung.com/java-generics
- https://www.geeksforgeeks.org/comparable-vs-comparator-in-java/
- https://www.baeldung.com/java-comparator-comparable

